



ARA0110 PADRÕES DE PROJETO

Aula 10: Padrões GRASP





CONTEXT0





DESAFIOS

Modelar sistemas orientados a objetos trazem diversos desafios e dúvidas a respeito do nosso trabalho, simplesmente por não ser algo tão trivial do dia a dia.

- **Onde colocar responsabilidades?**
- **Quem deve fazer o quê?**
- **Como evitar código frágil?**

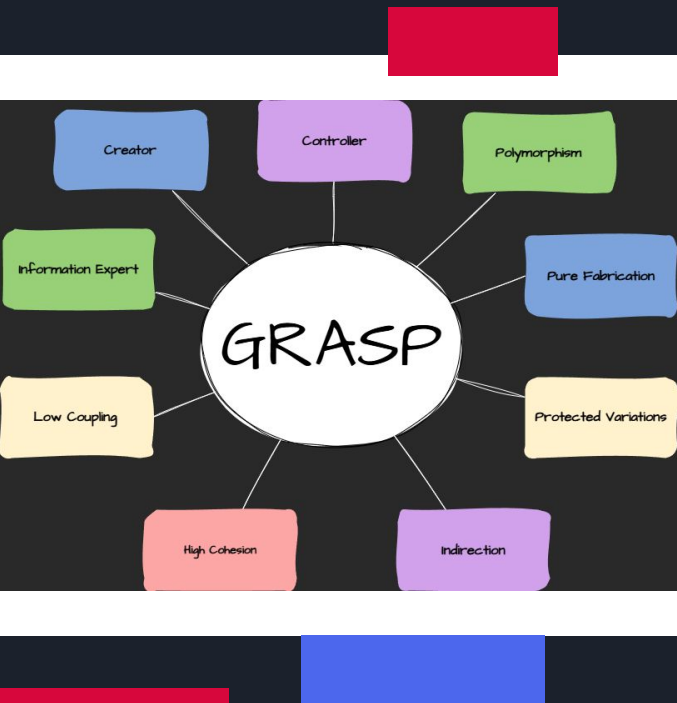
FOCO DA MODELAGEM

Melhor manutenção

Alta coesão

Baixo acoplamento

Código mais evolutivo



PADRÕES GRASP

O GRASP (General Responsibility Assignment Software Patterns) são diretrizes fundamentais para atribuir responsabilidades a classes e objetos na programação orientada a objetos.

Esses padrões focam em criar sistemas orientados a objetos com **alta coesão**, **baixo acoplamento** e código fácil de **manter**, **reutilizar** e **estender**.



CRIAÇÃO

O padrão foi proposto por Craig Larmer em 1997, com a finalidade de ser uma ferramenta pedagógica para ensinar o raciocínio por trás da orientação a objetos. Diferente dos padrões de projetos do GoF, que focam em soluções de código específicas, o GRASP foca em design de estruturas dos componentes orientados a objeto.

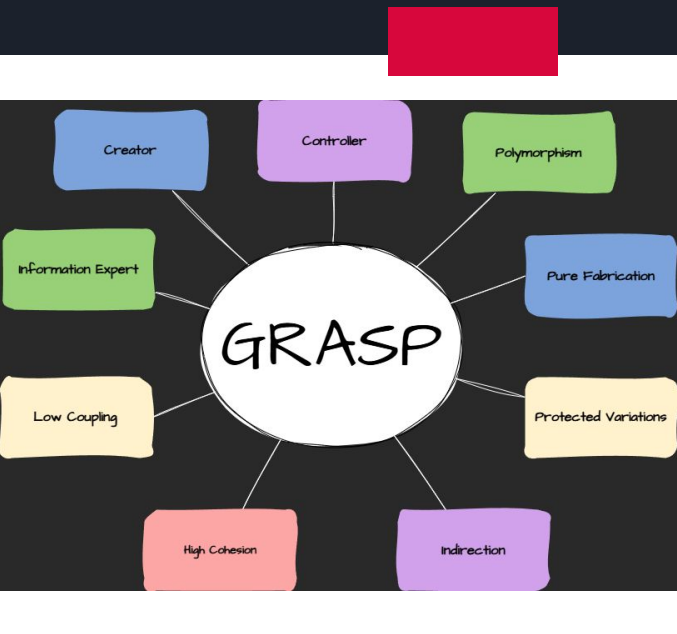


HISTÓRICO

Na época, a UML se tornava o padrão de modelagem, porém Larmer percebeu que saber desenhar diagramas não era o mesmo que projetar bons sistemas. Com isso, ele identificou que a maior dificuldade dos desenvolvedores era na atribuição de responsabilidades. Isso posto, ele publicou o livro *Applying UML and Patterns* (Utilizando UML e Padrões).

FILOSOFIA

- **Fundamentação em Responsabilidades:** Larmar se baseou no conceito de *Responsibility-Driven Design* (RDD), criado por Rebecca Wirfs-Brock, que foca na **colaboração** entre objetos
- **Abstração do GoF:** Enquanto os 23 padrões de projeto GoF são **receitas prontas**, Larmar queria ensinar os **ingredientes**. Os 9 princípios GRASP explicam o porquê dos padrões GoF funcionarem
- **Detalhe:** O GRASP não tecnicamente um conjunto de “novas” invenções, mas sim a formalização de **boas práticas** que já eram usadas por especialistas de Smalltalk e C++ nos anos 80, organizadas de forma lógica para novos engenheiros de software



PRINCÍPIOS

Os princípios GRASP não são uma lista solta de padrões, mas sim uma organização lógica em duas categorias principais baseadas no nível de complexidade e maturidade do design. A divisão consiste em padrões **fundamentais** e **avançados**, com o intuito de lidar com problemas de design partindo de situações simples até às mais complexas.



PADRÕES FUNDAMENTAIS

Estes são os pilares para quem está começando a modelar o sistema. Eles resolvem o problema imediato de “onde colocar este método?”:

- **Especialista na Informação:** O ponto de partida (onde está o dado?)
 - **Criador:** Define a origem dos objetos
 - **Baixo Acoplamento:** A meta de sustentabilidade da classe
 - **Alta Coesão:** A meta de clareza da classe
 - **Controlador:** A porta de entrada para a lógica do sistema
- 



PADRÕES AVANÇADOS

Estes padrões entram em jogo quando os padrões fundamentais começam a gerar classes “inchadas” ou sistemas rígidos demais:

- **Polimorfismo:** Resolve variações de comportamentos sem usar **if/else** ou **switch**
- **Fabricação Pura:** Quando a lógica não pertence a nenhum objeto real do negócio (salvar no banco de dados)
- **Indireção:** Cria uma ponte para evitar que duas classes se conheçam diretamente
- **Variações Protegidas:** Cria um “ponto de estabilidade” para que mudanças em uma parte não quebrem o resto





CENÁRIO DE EXEMPLO

Vamos utilizar como exemplo um sistema de vendas simples, na qual temos:

- Pedido
- Itens do pedido
- Produtos
- Cálculos do pedido

PRODUTO

```
public class Produto {  
  
    private final String nome;  
    private final double preco;  
  
    public Produto(String nome, double preco) {  
        this.nome = nome;  
        this.preco = preco;  
    }  
  
    public String getNome() {  
        return this.nome;  
    }  
  
    public double getPreco() {  
        return this.preco;  
    }  
}
```

ITEMPEDIDO

```
public class ItemPedido {  
  
    private Produto produto;  
    private int quantidade;  
  
    public ItemPedido(Produto produto, int quantidade) {  
        this.produto = produto;  
        this.quantidade = quantidade;  
    }  
  
    public Produto getProduto() {  
        return this.produto;  
    }  
  
    public int getQuantidade() {  
        return this.quantidade;  
    }  
}
```

PEDIDO

```
public class Pedido {  
  
    private final List<ItemPedido> itens;  
  
    public Pedido() {  
        this.itens = new ArrayList<>();  
    }  
  
    public List<ItemPedido> getItens() {  
        return this.itens;  
    }  
  
    public Pedido adicionar(ItemPedido item) {  
        this.itens.add(item);  
  
        return this;  
    }  
}
```



PADRÕES FUNDAMENTAIS





ESPECIALISTA NA INFORMAÇÃO

Neste princípio, a responsabilidade deve ser dada a quem possui os dados. Na prática, isso significa que, quando você precisa calcular o total de uma venda, a classe **Pedido** (que conhece os itens) deve fazer isso, e não uma classe externa. Dessa forma, mantemos a lógica próxima aos dados, facilitando a manutenção.

EXEMPLO RUIM

```
public class PedidoService {  
  
    public double calcularTotal(Pedido pedido) {  
        double total = 0;  
        for(ItemPedido item : pedido.getItems())  
            total += item.getProduto().getPreco() * item.getQuantidade();  
  
        return total;  
    }  
}
```

- Pedido tem os dados, mas não calcula
- Violação clara do princípio



CORREÇÃO

```
public class ItemPedido {  
  
    public double calcularSubTotal() {  
        return this.produto.getPreco() * this.quantidade;  
    }  
}  
  
public class Pedido {  
  
    public double calcularTotal() {  
        double total = 0;  
        for(ItemPedido item : this.itens)  
            total += item.calcularSubTotal();  
  
        return total;  
    }  
}
```



CRIADOR

Neste princípio, definimos de forma clara quem deve criar a instância de um objeto. Por exemplo, a classe **Pedido** deve criar objetos **ItemPedido** se ela os agrega, os contém ou tem os dados para inicializá-los.

Com isso, mantemos uma organização do ciclo de vida dos objetos e evita que qualquer classe saia criando instâncias sem critério.



EXEMPLO RUIM

```
public class PedidoService {  
  
    public ItemPedido criarItem(Produto produto, int quantidade) {  
        return new ItemPedido(produto, quantidade);  
    }  
}
```



CORREÇÃO

```
public class Pedido {  
  
    private final List<ItemPedido> itens;  
  
    public Pedido() {  
        this.itens = new ArrayList<>();  
    }  
  
    public void adicionarItem(Produto produto, int quantidade) {  
        ItemPedido item = new ItemPedido(produto, quantidade);  
        itens.add(item);  
    }  
}
```





BAIXO ACOPLAMENTO

Neste princípio, devemos manter as classes o mais independentes possível. Uma classe não deve conhecer detalhes internos de muitas outras, ou seja, quanto menos setas ligando classes em um diagrama melhor. Isso traz flexibilidade para o uso das classes pois, se você mudar uma classe, não quebra o sistema inteiro.

EXEMPLO RUIM

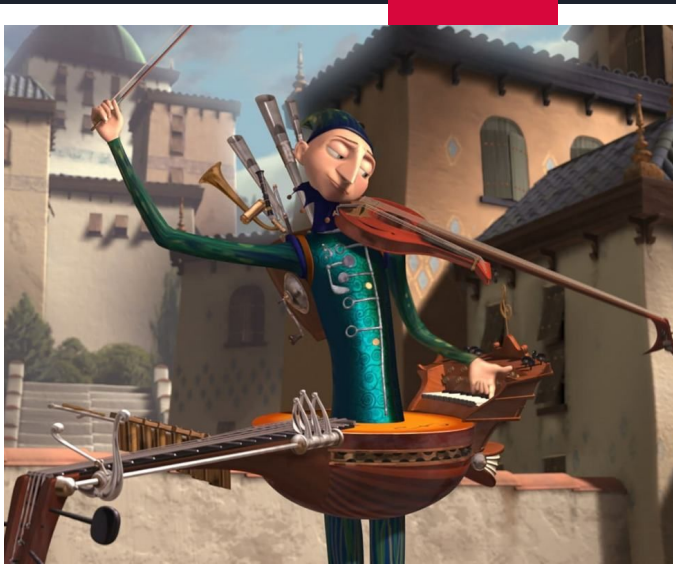
```
public class Pedido {  
  
    private MySQLConnection conn;  
  
    public Pedido(MySQLConnection conn) {  
        this.conn = conn;  
    }  
}
```



CORREÇÃO

```
public interface PedidosRepository {  
  
    void salvar(Pedido pedido);  
  
}  
  
public class PedidoService {  
  
    private PedidosRepository repository;  
  
    public PedidoService(PedidosRepository repository) {  
        this.repository = repository;  
    }  
  
    MySQLConnection conn = new MySQLConnection();  
    PedidosRepository repository = new MySQLPedidosRepository(conn);  
    PedidoService service = new PedidoService(repository);  
}
```





ALTA COESÃO

Neste princípio, uma classe deve fazer apenas uma coisa e fazê-la bem. Por exemplo, se uma classe trata de **Email**, ela não deve validar **CPF**. O código deve sempre ser focado e funcionalmente relacionado.

Isso facilita o entendimento e a reutilização do código.

EXEMPLO RUIM

```
public class Pedido {
```

```
    public void salvarNoBanco() {}
```

```
    public void enviarEmail() {}
```

```
    public void calcularTotal() {}
```

```
}
```

Infraestrutura

Comunicação

Regra de negócio



CORREÇÃO

```
public class Pedido {  
  
    public void calcularTotal() {}  
  
}  
  
public class PedidosRepository {  
  
    public void salvarNoBanco(Pedido pedido) {}  
  
}  
  
public class EmailService {  
  
    public void enviarEmail(Email email) {}  
  
}
```

```
public class PedidoService {  
  
    private PedidoRepository repository;  
    private EmailService emailService;  
  
    public void processar(Pedido pedido) {  
  
    }  
}
```



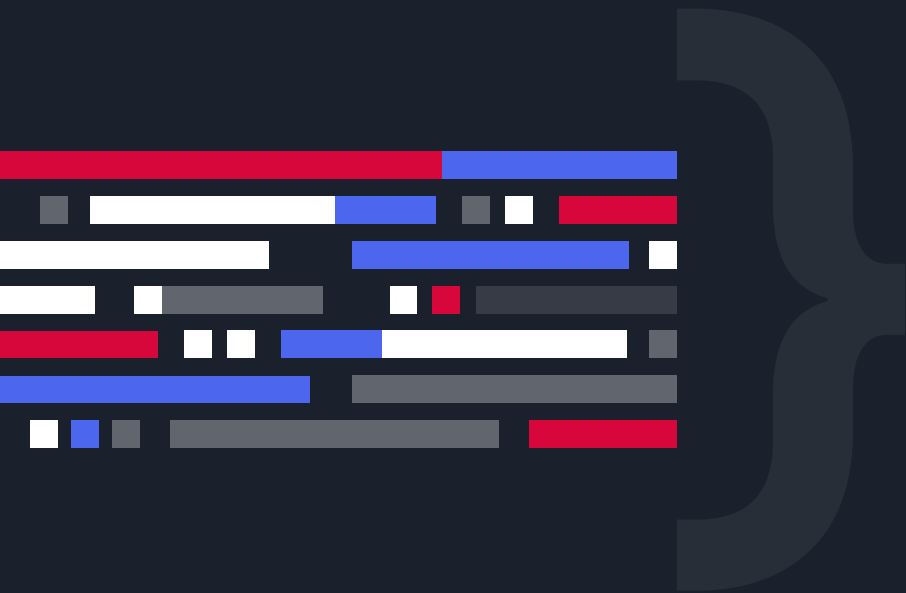


CONTROLADOR

Neste princípio, temos um componente que recebe e coordena uma operação do sistema, organizando a interação entre outros componentes. Por exemplo, podemos ter uma classe **ProcessadorVenda** que recebe o clique do botão “Finalizar” e delega as ações para os objetos de negócio. Isso permite separarmos a lógica de apresentação da lógica de negócio da aplicação.

EXEMPLO

```
public class ProcessarPedidosController {  
  
    private PedidoService service;  
    private ProdutorMensageria produtor;  
  
    public void handle(Pedido pedido) {  
        PedidoProcessado processado = this.service.processar(pedido);  
        if(processado.sucesso())  
            this.produtor.publicar(processado);  
    }  
}
```



TIPOS DE CONTROLADOR

- **Fachada do sistema:** encapsulam a orquestração de componentes para desempenhar operações.
- **Controlador de caso de uso:** contém os componentes necessários para atender a um cenário específico.

PADRÕES AVANÇADOS





POLIMORFISMO

Neste princípio, tratamos variações de comportamento com base em tipos de objetos. Ao invés de utilizarmos **`if(tipo == "Pix")`** ou **`if(tipo == "Cartão")`**, criamos uma interface **Pagamento** e deixamos cada classe implementar seu método. Isso nos permite adicionar novos comportamentos sem alterar o código existente.

EXEMPLO RUIM

```
public class PedidoService {  
  
    public void processar(Pedido pedido) {  
        if (pedido.tipoPagamento().equals("PIX")) {  
            this.processarPix(pedido.dadosPagamento());  
        } else if (pedido.tipoPagamento().equals("CREDITO")) {  
            this.processarCartaoCredito(pedido.dadosPagamento());  
        } else {  
            this.processarCartaoDebito(pedido.dadosPagamento());  
        }  
    }  
}
```

- Alto acoplamento
- Difícil de estender
- Viola o princípio Aberto/Fechado



CORREÇÃO

```
public interface Pagamento {  
    void processar(DadosPagamento dados);  
}  
  
public class PagamentoPix implements Pagamento {  
  
    public void processar(DadosPagamento dados) {  
        // Processa o PIX  
    }  
}  
  
public class PagamentoCredito implements Pagamento {  
  
    public void processar(DadosPagamento dados) {  
        // Processa o cartão de crédito  
    }  
}
```

```
public class PedidoService {  
  
    private final Pagamento pagamento;  
  
    public PedidoService(Pagamento pagamento) {  
        this.pagamento = pagamento;  
    }  
  
    public void processar(Pedido pedido) {  
        this.pagamento.processar(pedido.dadosPagamento());  
    }  
}
```



FABRICAÇÃO PURA



Na Fabricação Pura, criamos uma classe que não representa algo do mundo real para manter o design limpo. Para entender isso, imagine um componente responsável por salvar dados de um cliente no banco de dados: um repositório não existe no mundo real, mas ele é usado para melhorar a coesão, reduzir o acoplamento e organizar responsabilidades dentro da aplicação.

EXEMPLO

```
public class PedidosRepository {  
  
    public void salvarNoBanco(Pedido pedido) {}  
  
}  
  
public class PedidoService {  
  
    private PedidoRepository repository;  
  
    public void processar(Pedido pedido) {  
  
    }  
}
```

INDIREÇÃO



Neste princípio, colocamos um objeto intermediário entre dois outros para eliminarmos seu acoplamento. Essa classe intermediária pode representar um **Adaptador** ou **Mediador**, onde uma classe A fala com o intermediário, que fala com uma classe B. Isso quebra o vínculo direto entre os componentes, permitindo que ambos evoluam separadamente.

EXEMPLO

```
public class PedidoService {  
  
    private PagamentoGateway gateway;  
  
    public PedidoService(PagamentoGateway gateway) {  
        this.gateway = gateway;  
    }  
  
    public void pagar(Pedido pedido) {  
        this.gateway.processar(pedido);  
    }  
}
```



VARIAÇÕES PROTEGIDAS

Neste princípio, identificamos pontos de mudança provável e criamos uma interface estável ao redor delas. Por exemplo, se o fornecedor de envio de SMS pode mudar, crie uma interface **SmsService** e faça o sistema usar a interface já que, como esperado, quem estiver por trás do envio de SMS pode mudar por qualquer motivo.

EXEMPLO RUIM

```
public class PagamentoService {  
  
    private StripeGateway gateway;  
  
    public void processar(DadosPagamento dados) {  
        this.gateway.processar(dados);  
    }  
}
```



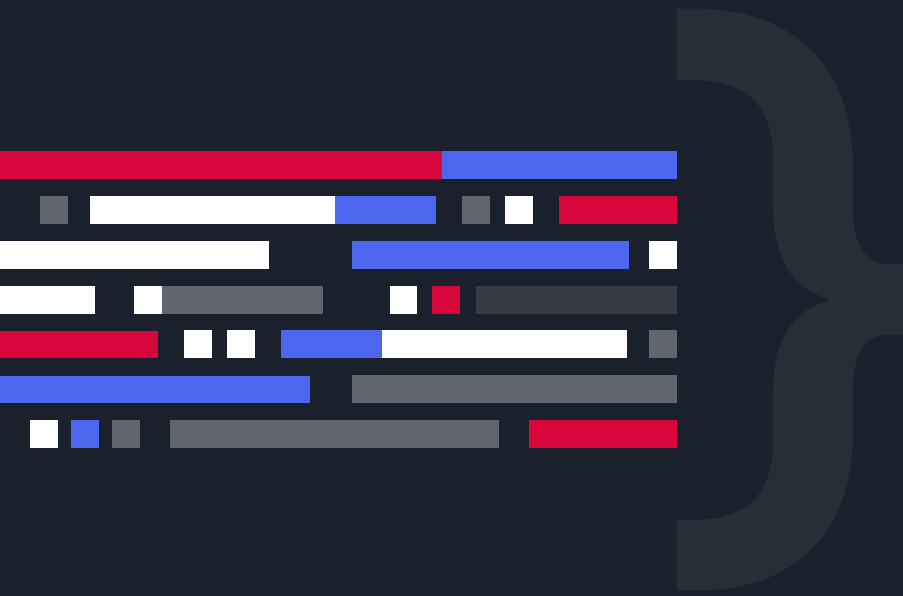
CORREÇÃO

```
public interface PagamentoGateway {  
    void processar(DadosPagamento dados);  
}  
  
public class StripeGateway implements PagamentoGateway {  
    public void processar(DadosPagamento dados) {}  
}  
  
public class MercadoPagoGateway implements PagamentoGateway {  
    public void processar(DadosPagamento dados) {}  
}  
  
public class PagamentoService {  
    private PagamentoGateway gateway;  
  
    public void processar(DadosPagamento dados) {  
        this.gateway.processar(dados);  
    }  
}
```



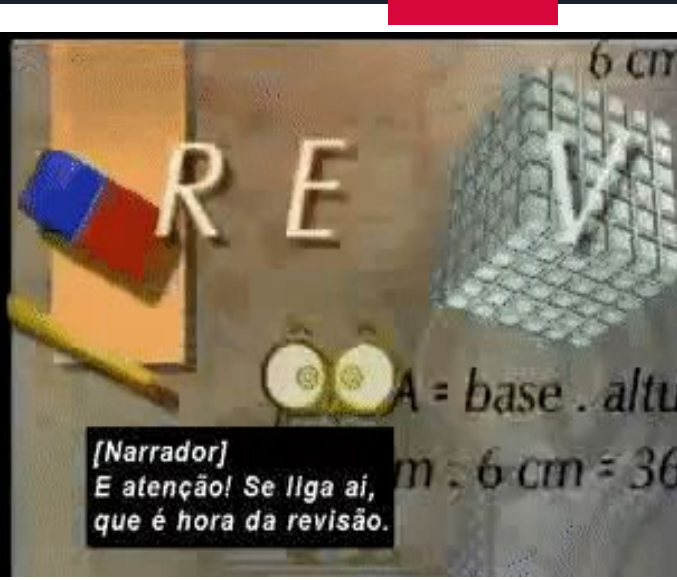
CONCLUSÃO





PRINCÍPIOS DE DESIGN

O foco dos padrões GRASP não está concentrado diretamente na implementação de um componente de fato, mas sim em como ele interage e como ele se comporta dentro de um sistema complexo, permitindo obter benefícios de boas modelagens que favoreçam manutenção e extensão de funcionalidades.



RECAPITULANDO

- Definição
- Benefícios dos princípios
- Padrões fundamentais
- Padrões avançados
- Exemplos

OBRIGADO !



"FALICEU!"

José Adilson Rodrigues dos Santos, a.k.a. Maguila